

SmartCare v0.1 - Initial Engineering Brief and AI Activity Card

A2 Case Study Stage 1 student resource

SmartCare scenario

SmartCare Community Clinic currently uses spreadsheets and paper records to manage patients and appointments. The client says: 'We need software to help manage patients, practitioners and appointments.' This is not yet a complete specification.

Initial Engineering Brief

1. Problem summary

As it stands, SmartCare Community Clinic uses a scattered combination of paper documents, spreadsheet programs, and manual procedures for appointment and scheduling purposes. This has resulted in friction within the process, causing duplicate bookings, misplaced patient documents, failure to log canceled appointments, and no knowledge of the availability of the doctors. A system is needed by the clinic administration that will be able to centralize patient information, scheduling of practitioners, and manage appointments. It should still be practical and easy to use in a community clinic without being complicated like a hospital system.

2. Initial stakeholders

Stakeholder	Possible need
Clinic Receptionists / Administrative Staff	Fast search for patient records, easy booking without duplicate slots, and a clear daily calendar view.
Healthcare Practitioners (GPs)	Clear visibility of daily appointments, access to reliable patient appointment histories, and accurate scheduling.
Patients	Reliable booking confirmations, reduced waiting times, and assurance of data accuracy and privacy.
Clinic Management	Basic operational summaries (daily volumes, cancellation trends) and a low-maintenance, cost-effective system.

3. Initial features

Feature	Confirmed or provisional?	Why?
Patient Record Management	Confirmed	Explicitly stated in the client brief as an immediate core requirement.
Practitioner Availability Tracker	Confirmed	Directly addresses the problem of limited practitioner visibility.
Appointment Booking & Duplicate Prevention	Confirmed	Needed to resolve current duplicate booking errors.
Automated SMS / Email Notifications	Provisional	Beneficial for reducing missed visits, but not explicitly requested

		for prototype v0.1.
Basic Operational Reporting	Confirmed	Explicitly requested by management to evaluate clinic appointment metrics.

4. Questions for the client

1. What are the clinic's standard operating hours and consultation slot durations (e.g., 15 vs 30 minutes)?
2. What fields are mandatory to register a new patient (e.g., full name, phone number, Medicare ID)?
3. How should cancelled appointments be recorded—permanently removed or archived as "Cancelled" for audit history?
4. How many practitioners work at the clinic, and do they have varying weekly shifts?
5. What exact figures or summaries are expected in the weekly/monthly operational reports?

5. What we do not yet know

1. The exact data storage format preferred for future persistence (e.g., CSV, JSON, or lightweight SQLite).
2. The rules governing practitioner leave, emergency slots, and double-practitioner consultations.
3. Specific user access levels and permissions (e.g., should receptionists access medical histories or only booking data?).

AI Activity Card - Ask, Check, Explain

Before AI

What do I think the code does?

It creates hardcoded appointments or takes inputs and appends them as dictionary objects into an in-memory list, then loops through to print each appointment.

What problems can I already identify?

It loses all data when closed, allows duplicate appointments for the same doctor and time, and lacks proper input sanitisation (such as whitespace stripping or datetime validation).

AI request

Act as a tutor. Explain this code and identify potential problems. Do not provide a complete replacement. Ask me questions that help me reason about the solution.

Can you help me create a simple beginner-friendly Python function that stores patient name, practitioner name and appointment time. I would like you to not include using a database or

Evaluate

Suggestion	Useful	Unclear	Incorrect	Out of scope
Validate empty or whitespace-only patient names	✓			
Integrate an SQL database				✓
Use strict ISO datetime parsing	✓			
Check practitioner schedule clashes before booking	✓			

Decide

For each significant suggestion: Accept / Modify / Reject / Keep unverified.

Verify

- Run the code
- Test normal input
- Test unusual input
- Compare with requirements
- Ask tutor/peer
- Check documentation

Explain

Can I explain the final code without reading the AI response?

Yes. The code uses a global list containing dictionary objects. The `book_appointment` function validates input strings, iterates through existing records to check for matching doctor and time values, appends valid entries, and prints them sequentially.

What do I still need to understand?

How to refactor this procedural structure into Object-Oriented classes (Patients, Practitioners, Appointments) and implement file persistence in future stages.